Architecture Technique — Sécurité Convergée IoT/IA pour Bâtiments Sensibles

> ***Réalisé par :*** *Ryan ZERHOUNI*
> ***Version :*** *0.1 — Document de travail (avant développement)*

Table des matières

1. Vue d'ensemble du système
2. Couche IoT — Sources de données terrain
3. Couche Protocoles & Gateway
4. Couche Sécurité — TLS sans PKI & PQC
5. Couche Middleware — Normalisation & Corrélation
6. Couche Streaming & Stockage
7. Couche IA — Intelligence Comportementale
8. Couche Présentation — Dashboard SOC
9. Flux de données de bout en bout
10. Stack Technologique
11. Modèle de données
12. Scénarios d'attaque & réponses attendues
13. Défis techniques & points à trancher

---

1. Vue d'ensemble du système

Le système est organisé en **7 couches fonctionnelles** qui communiquent de façon unidirectionnelle (terrain → décision), avec retour de commandes depuis le dashboard.

```
┌──────────────────────────────────────────────────────────────┐
│                    [8] DASHBOARD SOC / UI                    │
│               Alertes · Scores · Cartes · Logs               │
└──────────────────────────────┬───────────────────────────────┘
                               │ WebSocket / REST
┌──────────────────────────────▼───────────────────────────────┐
│               [7] MOTEUR IA — Analyse comportementale        │
│         Scoring Normal / Suspect / Critique · Anomalies      │
└──────────────────────────────┬───────────────────────────────┘
                               │ Events enrichis
┌──────────────────────────────▼───────────────────────────────┐
│          [6] STREAMING & STOCKAGE — Kafka · TimescaleDB      │
│               File d'événements · Historique · Logs         │
└──────────────────────────────┬───────────────────────────────┘
                               │ Données normalisées
┌──────────────────────────────▼──────────────────────────────┐
│        [5] MIDDLEWARE — Node-RED · Mosquitto MQTT Broker     │
│          Agrégation · Normalisation · Corrélation physique/cyber│
└──────────────────────────────┬──────────────────────────────┘
                               │ Flux sécurisé (TLS/PQC)
┌──────────────────────────────▼───────────────────────────────┐
│        [4] SÉCURITÉ — TLS sans PKI · PQC (X25519MLKEM768)    │
│       Tunnels hybrides · Logs inviolables · Auth mutuelle    │
└──────────────────────────────┬───────────────────────────────┘
                               │
┌──────────────────────────────▼───────────────────────────────┐
│            [3] GATEWAY / EDGE — Collecte locale              │
│          Filtrage · Pre-processing · Buffer offline          │
└──────────┬──────────────────────────────┬────────────────────┘
           │ MQTT                         │ HTTP
┌──────────▼──────────────────────────────▼────────────────────┐
│               [2] PROTOCOLES DE COMMUNICATION                │
└──────────┬──────────┬──────────┬──────────┬──────────────────┘
           │          │          │          │
┌──────────▼─┐ ┌──────▼─────┐ ┌──▼─────┐ ┌──▼──────────────────┐
│Lecteurs    │ │Capteurs    │ │Detect. │ │Caméras & capteurs env.│
│ badges     │ │portes      │ │mouvt.  │ │(temp, humidité, vidéo)│
└────────────┘ └────────────┘ └────────┘ └─────────────────────┘
                    [1] COUCHE IoT — TERRAIN
```

2. Couche IoT — Sources de données terrain (SIMULÉES)

> ***Contexte déploiement :*** *aucun matériel physique. Tous les capteurs sont émulés par un* ***simulateur Python*** *tournant sur la machine hôte. Chaque type de capteur est un processus (ou thread) qui génère des événements selon un modèle probabiliste configurable, publie en MQTT, et peut injecter des scénarios d'attaque à la demande.*

2.1 Architecture du simulateur

```
simulator/
├── main.py                  ← orchestrateur : démarre tous les agents
├── config.yaml              ← topologie du bâtiment (zones, portes, users)
├── agents/
│   ├── badge_agent.py       ← génère des accès badges (normaux + anomalies)
│   ├── door_agent.py        ← génère états portes (open/close/forced)
│   ├── motion_agent.py      ← génère détections de mouvement
│   ├── camera_agent.py      ← génère métadonnées vidéo (pas de vrai flux)
│   └── network_agent.py     ← génère événements réseau (trafic, scans)
├── scenarios/
│   ├── normal_day.py        ← journée normale (baseline IA)
│   ├── intrusion_physical.py← porte forcée sans badge
│   ├── hybrid_attack.py     ← intrusion physique + scan réseau simultané
│   └── tailgating.py        ← badge OK + mouvement double
└── mqtt_client.py           ← client MQTT partagé (paho-mqtt)
```

2.2 Modèle de simulation par agent

Badge Agent — génère des accès selon une distribution réaliste :

```
# Comportement normal : gaussienne autour des horaires de travail
access_time ~ Normal(μ=9h00, σ=45min)  # arrivée matin
access_time ~ Normal(μ=18h00, σ=30min) # départ soir

# Anomalie injectable : accès à 3h17 du matin
# Anomalie injectable : badge révoqué, badge inconnu
```

Network Agent — génère du trafic réseau simulé (pas de vrai trafic capturé) :

```
# Normal : volume constant par IP, ports standards
# Anomalie : scan de ports (SYN burst), exfiltration (volume élevé sortant)
```

2.3 Format de message émis (identique au cas réel)

Les agents publient exactement le même format JSON qu'un vrai capteur ferait — Node-RED et la couche IA ne voient pas la différence.

Champ émis	Type	Description
`badge_id`	string	Identifiant unique du badge
`user_id`	string	Utilisateur associé
`timestamp`	datetime	Horodatage ISO 8601
`location_id`	string	Zone / porte ciblée
`access_result`	enum	`GRANTED` / `DENIED` / `TIMEOUT`

Topics MQTT publiés par le simulateur :

```
building/B1/zone/{zone_id}/badge/{reader_id}      ← badge_agent
building/B1/zone/{zone_id}/door/{door_id}         ← door_agent
building/B1/zone/{zone_id}/motion/{detector_id}   ← motion_agent
building/B1/network/flow                          ← network_agent
building/B1/network/alert                         ← network_agent (anomalies)
```

---

3. Couche Protocoles & Gateway (LOGICIELLE)

> ***Contexte déploiement :*** *pas de gateway physique. Le gateway est un* ***service Python conteneurisé*** *qui joue le rôle d'intermédiaire entre le simulateur et le middleware. Dans un déploiement réel, ce service tournerait sur un Raspberry Pi ; ici il tourne dans un container Docker sur le même hôte.*

3.1 Protocoles supportés

Protocole	Usage dans la simulation	Bibliothèque Python
MQTT	Communication simulateur → gateway	`paho-mqtt`
HTTP	API REST gateway → middleware	`httpx` / `FastAPI`

3.2 Gateway logiciel — responsabilités

```
[Simulateur Python] ──MQTT──▶ [Gateway Service]
                                        │
                                        ├── Validation du format JSON (schema)
                                        ├── Déduplication (évite les doublons)
                                        ├── Buffer en mémoire (queue Python)
                                        ├── Chiffrement TLS sortant (X25519MLKEM768)
                                        └── Publication vers Mosquitto Broker
```

Technologie : service Python (`asyncio` + `paho-mqtt`), containerisé Docker.

> ***Note :*** *dans cette version simulée, le gateway et le simulateur tournent sur le même hôte. La couche TLS s'applique quand même sur les connexions réseau locales (localhost avec TLS) pour valider l'implémentation crypto dans des conditions réalistes.*

---

4. Couche Sécurité — TLS sans PKI & PQC

4.1 Architecture TLS sans PKI — Adaptation simulation logicielle

> ***Contexte déploiement :*** *pas de Secure Element ni de HSM physique. Les clés sont générées et stockées dans des* ***fichiers protégés sur le système de fichiers*** *local (permissions restreintes, dossier chiffré). Dans un déploiement réel sur matériel, ces mêmes clés seraient gravées en SE/HSM à la fabrication. L'architecture cryptographique reste identique — seul le stockage physique change.*

Équivalences simulation / production :

Élément	Production (matériel)	Simulation (PC hôte)
Stockage clé privée	Secure Element / HSM (non extractible)	Fichier `.pem` chiffré (AES-256), permissions 600
Provisionnement	Gravé en usine	Script de génération au premier démarrage
Anti-extraction	Matériel (impossible physiquement)	Contrôle d'accès OS + chiffrement du fichier
Allow-list	Identique	Identique (fichier JSON de certificats autorisés)

Structure des fichiers de clés (simulation) :

```
security/
├── ca/
│   ├── ca.crt                  ← certificat racine (hybride ECC-hybrid-MLDSA5)
│   └── ca.key                  ← clé privée racine (hybride, chiffrée)
├── gateway/
│   ├── gateway.crt             ← certificat d'identité du gateway logiciel
│   └── gateway.key             ← clé privée hybride (ECC-hybrid-MLDSA5, perm. 600)
├── middleware/
│   ├── middleware.crt          ← certificat d'identité du middleware
│   └── middleware.key          ← clé privée hybride (ECC-hybrid-MLDSA5)
└── allowlist.json              ← liste des certificats autorisés (mTLS sans PKI)
```

Flux d'authentification mutuelle (inchangé vs production) : 1. Le simulateur/gateway présente son certificat → le middleware valide via l'allow-list 2. Le middleware présente son certificat → le gateway valide 3. Session TLS 1.3 établie avec X25519MLKEM768 comme groupe de clés

4.2 Cryptographie Post-Quantique (PQC)

Algorithmes utilisés :

Fonction	Algorithme hybride retenu	Détail	Standard NIST
Échange de clés (KEM)	**X25519MLKEM768**	X25519 (ECDH classique) + ML-KEM-768 (Kyber niveau 3, 192-bit sec.)	FIPS 203 + RFC 8422
Authentification / Signature	**ECC-hybrid-MLDSA5**	ECDSA P-384 (classique) + ML-DSA niveau 5 (256-bit sec. équiv. AES-256)	FIPS 204
Hash / intégrité	**SHA-3 / SHAKE-256**	Résistant quantique nativement (construction sponge)	FIPS 202

Principe du tunnel hybride X25519MLKEM768 :

```
Client (Gateway)                          Serveur (Middleware)
      |                                          |
      |── ClientHello (TLS 1.3) ────────────────►|
      |   key_share:                             |
      |     X25519   : PK_x25519_client          |
      |     ML-KEM-768 : PK_mlkem_client         |
      |                                          |
      |◄── ServerHello ──────────────────────────|
      |   key_share:                             |
      |     X25519   : PK_x25519_server          |
      |     ML-KEM-768 : CT_mlkem (ciphertext)   |
      |                                          |
      |  Secret partagé final :                  |
      |  SS = KDF(SS_x25519 ‖ SS_mlkem)          |
      |  → Si x25519 cassé : SS_mlkem tient      |
      |  → Si ML-KEM cassé : SS_x25519 tient     |
      |                                          |
      |═══ Session TLS 1.3 chiffrée (AES-256-GCM) ═══════|
```

Principe du hybride ECC-hybrid-MLDSA5 pour la signature :

```
Signature d'un log ou d'un certificat :
  sig_final = (sig_ECDSA_P384 ‖ sig_MLDSA5)

Vérification :
```

```
valide si ET SEULEMENT SI sig_ECDSA_P384 ET sig_MLDSA5 sont tous les deux valides
→ double signature → sécurité maximale sur 15 ans face aux menaces quantiques
```

Choix de ML-DSA niveau 5 (ECC-hybrid-MLDSA5) : ECC-hybrid-MLDSA5 offre le niveau de sécurité 5 (équivalent AES-256), le plus élevé du standard FIPS 204. Justifié ici car les logs signés doivent rester légalement incontestables sur une durée de 15 ans, pendant laquelle la puissance de calcul quantique évoluera de façon imprévisible.

4.2.4 Optimisation des performances : Session Resumption hybride (PSK+DHE)

Pour compenser l'overhead calculatoire et la taille des messages du handshake PQC complet (ML-KEM-768), le système implémente le mécanisme de **TLS 1.3 Session Resumption** en mode hybride : * **Handshake Initial (Full)** : Échange de clés complet **X25519 + ML-KEM-768**. Après authentification mutuelle, un ticket de session (PSK - Pre-Shared Key) est généré. * **Reconnexions (Resumption)** : Utilisation du **PSK** combiné à un échange **DHE éphémère (X25519)**.

> *[!IMPORTANT] **Héritage de la résistance quantique** : La reprise de session ne perd pas sa sécurité PQC. Le PSK utilisé lors de la reprise est directement dérivé du secret partagé établi via ML-KEM-768 lors du handshake initial. Même si l'échange éphémère de la reconnexion est uniquement classique (X25519), le secret global reste protégé par l'entropie quantique du PSK "parent".*

Bénéfices : - **Performance** : Réduction de 90% de l'overhead PQC sur les reconnexions. - **Perfect Forward Secrecy (PFS)** : L'ajout de l'échange X25519 sur chaque reconnexion garantit que le vol physique d'un ticket de session ne permet pas de déchiffrer les sessions passées ou futures.

4.3 Protection des logs

- **Logs inviolables :** chaque entrée est signée avec le hybride ECC-hybrid-MLDSA5 → toute modification est détectable, y compris par un adversaire quantique futur
- **Chiffrement des logs au repos :** avec la clé de session issue de X25519MLKEM768 → protégés contre le "harvest now, decrypt later"
- **Horodatage qualifié :** timestamp signé par une TSA (Time Stamping Authority) pour valeur légale
- **Durée de garantie : 15 ans** — justifiée par le choix ML-DSA niveau 5 (ECC-hybrid-MLDSA5)

4.4 Architecture de Segmentation — Double Tunnel Proxy

Afin de garantir une isolation stricte et une gestion agnostique de la cryptographie, le flux entre la zone IoT et le Middleware est segmenté par un dispositif de **double proxy** agissant comme terminaisons PQC (PQC Terminations) :

1. **Forward Proxy (Côté Gateway/Edge)** : Intercepte les flux locaux (MQTT/HTTP) et les encapsule dans le tunnel TLS/PQC sortant.
2. **Reverse Proxy (Côté Middleware/Cloud)** : Termine le tunnel PQC, vérifie l'identité du client et redirige le trafic déchiffré vers les services internes (Mosquitto Broker, Node-RED).

Avantages stratégiques : - **Agnosticisme applicatif** : Les applications (simulateur, Node-RED, Kafka) n'ont pas besoin de supporter nativement les bibliothèques PQC ; elles communiquent en clair ou via TLS classique sur des interfaces locales sécurisées. - **Défense en profondeur** : Le tunnel PQC agit comme une couche de transport inviolable (Post-Quantum Secure Pipe), isolée de la logique métier. Cette architecture repose sur une segmentation réseau stricte (Docker Networks isolés), garantissant que le tunnel PQC est l'unique vecteur de communication autorisé entre le périmètre IoT et le périmètre Middleware. L'isolation est renforcée par l'utilisation de réseaux internes (`internal: true`) pour les périmètres IoT et Middleware. Seul le segment de transit entre les terminaux PQC dispose d'une interface réseau exposée, réduisant la surface d'attaque globale aux seules extrémités du tunnel.

---

5. Couche Middleware — Normalisation & Corrélation

5.1 Mosquitto MQTT Broker

Rôle : bus de messages central recevant tous les événements des gateways.

Topics MQTT structurés :

```
building/{building_id}/zone/{zone_id}/badge/{reader_id}
building/{building_id}/zone/{zone_id}/door/{door_id}
building/{building_id}/zone/{zone_id}/motion/{detector_id}
```

```
building/{building_id}/network/flow
building/{building_id}/network/alert
```

5.2 Node-RED — Orchestration & Normalisation

Rôle : plateforme de programmation visuelle par flux (flow-based programming) qui consomme les messages MQTT et les transforme en événements normalisés. Node-RED est particulièrement adapté à l'IoT : nœuds MQTT natifs, large écosystème de nœuds communautaires, et déploiement léger.

Architecture des flows Node-RED :

```
[Flow 1 — Ingestion MQTT multi-sources]

  [mqtt in] badge    ──┐
  [mqtt in] door     ──┤
  [mqtt in] motion   ──┼──▶ [switch] type ──▶ [function] parser ──▶ [function]
validate
  [mqtt in] camera   ──┤
  [mqtt in] network  ──┘

[Flow 2 — Normalisation & Enrichissement]

  [function] validate
      │
      ▼
  [function] normalize        ← mise au format Unified Event Schema
      │
      ▼
  [http request] GET user     ← appel API interne pour enrichir user_id → nom, zone
autorisée
      │
      ▼
  [function] enrich
      │
      ├──▶ [kafka out] topic: events.raw
      └──▶ [function] log_sign ──▶ [timescaledb out] logs signés

[Flow 3 — Corrélation physique / cyber]

  [kafka in] events.raw
      │
      ▼
  [function] time_window_buffer    ← fenêtre glissante de 10s par zone
      │
```

```
      ▼
  [function] correlate            ← détecte (badge + trafic réseau) dans la même
zone/fenêtre
      │
      ├── corrélation trouvée ──▶ [function] merge_events ──▶ [kafka out] events.raw
(enrichi)
      └── pas de corrélation  ──▶ pass-through

[Flow 4 — Alertes critiques temps réel]

  [kafka in] alerts.critical
      │
      ▼
  [switch] classification
      ├── CRITICAL ──▶ [http request] POST webhook SOC
      │              ▶ [email out] notification équipe
      │              ▶ [websocket out] dashboard
      └── SUSPECT  ──▶ [websocket out] dashboard (niveau 2)

[Flow 5 — Health monitoring des devices IoT]

  [inject] timer 30s ──▶ [http request] GET /devices/status
      │
      ▼
  [function] check_last_seen    ← device silencieux > seuil = alerte
      │
      └── device KO ──▶ [mqtt out] building/.../device/alert
```

Nœuds Node-RED utilisés :

Nœud	Source	Usage
`node-red-contrib-mqtt`	Core	Subscribe/Publish Mosquitto
`node-red-contrib-kafka`	npm communautaire	Produce/Consume Kafka
`node-red-contrib-postgresql`	npm communautaire	Écriture TimescaleDB
`node-red-contrib-http-request`	Core	Appels REST internes (enrichissement)

Nœud	Source	Usage
`node-red-contrib-websocket`	Core	Push temps réel vers dashboard
`function`	Core	Logique métier en JavaScript
`switch`	Core	Routage conditionnel
`inject`	Core	Déclencheurs temporels (health checks)
`debug`	Core	Supervision des flux en développement

5.3 Format d'événement normalisé (Unified Event Schema)

```
{
  "event_id": "uuid-v4",
  "event_type": "BADGE_ACCESS | DOOR_FORCED | MOTION_DETECTED | NETWORK_ANOMALY
        | ...",
  "source_layer": "PHYSICAL | CYBER",
  "timestamp": "ISO 8601",
  "building_id": "string",
  "zone_id": "string",
  "device_id": "string",
  "user_id": "string | null",
  "severity_raw": "INFO | WARNING | ALERT",
  "payload": { /* données brutes spécifiques à l'event_type */ },
  "correlated_events": ["event_id_1", "event_id_2"],
  "ai_score": null,           // rempli par la couche IA
  "ai_classification": null  // "NORMAL" | "SUSPECT" | "CRITICAL"
}
```

---

6. Couche Streaming & Stockage

6.1 Apache Kafka — Streaming temps réel

Rôle : file de messages haute performance entre le middleware et le moteur IA. Kafka s'appuie sur **Zookeeper** pour la coordination des brokers et la gestion de la configuration du cluster.

Topics Kafka :

Topic	Producteur	Consommateur	Rétention
`events.raw`	Middleware	IA Engine	7 jours
`events.enriched`	IA Engine	Dashboard, DB	30 jours
`alerts.critical`	IA Engine	SOC, Notif.	90 jours
`logs.signed`	Middleware	TimescaleDB	15 ans

6.2 TimescaleDB — Stockage série temporelle

Rôle : base de données optimisée pour les séries temporelles (extension PostgreSQL).

Tables principales :

```sql
-- Événements normalisés
CREATE TABLE events (
  event_id     UUID PRIMARY KEY,
  event_type   VARCHAR(50),
  source_layer VARCHAR(10),
  timestamp    TIMESTAMPTZ NOT NULL,
  building_id  VARCHAR(50),
  zone_id      VARCHAR(50),
  device_id    VARCHAR(50),
  user_id      VARCHAR(50),
  ai_score     FLOAT,
  ai_class     VARCHAR(10),
  payload      JSONB,
  signature    BYTEA  -- signature PQC du log
);
SELECT create_hypertable('events', 'timestamp');

-- Profils comportementaux utilisateurs (baseline IA)
CREATE TABLE user_profiles (
  user_id          VARCHAR(50) PRIMARY KEY,
  typical_zones    TEXT[],
  typical_hours    INT4RANGE[],
  avg_duration_s   FLOAT,
  last_updated     TIMESTAMPTZ
);
```

```sql
-- Scores de risque historiques
CREATE TABLE risk_scores (
  score_id    UUID PRIMARY KEY,
  entity_id   VARCHAR(50),  -- user_id ou device_id
  entity_type VARCHAR(10),
  score       FLOAT,
  timestamp   TIMESTAMPTZ NOT NULL
);
SELECT create_hypertable('risk_scores', 'timestamp');
```

---

7. Couche IA — Intelligence Comportementale

7.1 Pipeline de traitement

```
Kafka (events.raw)
    │
    ▼
[1] Feature Extraction
    │  → Heure, zone, user, durée, fréquence, delta réseau
    ▼
[2] Behavioral Baseline (modèle de référence par user/zone)
    │  → Comparaison avec profil historique (TimescaleDB)
    ▼
[3] Anomaly Scoring
    │  → Score 0.0 → 1.0 par dimension
    ▼
[4] Cross-Correlation (physique + cyber)
    │  → Fusion des signaux physiques et réseau dans une fenêtre temporelle
    ▼
[5] Risk Classification
    │  → NORMAL (< 0.3) | SUSPECT (0.3-0.7) | CRITICAL (> 0.7)
    ▼
Kafka (events.enriched) + TimescaleDB
```

7.2 Modèles utilisés

Modèle	Usage	Technologie
Isolation Forest	Détection d'anomalies non supervisée	scikit-learn
LSTM / Time Series	Modélisation des séquences d'accès temporels	PyTorch / Keras
Rule-based engine	Règles déterministes (porte forcée = CRITICAL)	Python / Node-RED
Fusion scorer	Combinaison scores physique + cyber	Python (poids appris)

7.3 Score de risque dynamique

Formule de fusion (à affiner) :

```
score_final = min(1.0, w1 × score_physique + w2 × score_cyber + w3 × score_corrélation)
```

- `w1, w2, w3` : poids appris par validation croisée (ex: $w3 = 0.2$)
- `score_corrélation` : variable binaire (1 si `correlated_events` est non vide, 0 sinon)

Classification :

```
score ∈ [0.0, 0.3)  →  NORMAL    (log uniquement)
score ∈ [0.3, 0.7)  →  SUSPECT   (alerte SOC niveau 2)
score ∈ [0.7, 1.0]  →  CRITICAL  (alerte immédiate, action automatique possible)
```

7.4 Dérive du modèle (Model Drift)

- Réentraînement périodique sur les 90 derniers jours d'événements validés
- Métriques surveillées : taux de faux positifs, taux de faux négatifs, distribution des scores
- A/B testing entre ancienne et nouvelle version du modèle avant déploiement

---

8. Couche Présentation — Dashboard SOC

8.1 Fonctionnalités du dashboard

- **Vue cartographique** du bâtiment : zones colorées selon le niveau de risque courant
- **Fil d'alertes temps réel** : événements triés par score, filtres par zone/type/sévérité
- **Fiche utilisateur** : historique d'accès, score de risque évolutif, événements associés
- **Fiche device** : état de chaque capteur IoT, santé, dernière communication
- **Timeline de corrélation** : visualisation des événements physiques + cyber sur un axe temporel commun
- **Logs auditables** : consultation des logs signés PQC, export pour investigation légale

8.2 Stack frontend

Composant	Technologie
Framework	React + TypeScript
Temps réel	WebSocket (Socket.io)
Cartographie	Leaflet.js / SVG floors
Charts	Recharts / D3.js
API backend	FastAPI (Python)

8.3 API backend (FastAPI)

```
GET  /api/events?zone=&from=&to=&class=
GET  /api/alerts/active
GET  /api/users/{user_id}/profile
GET  /api/devices
GET  /api/score/current
POST /api/alert/{alert_id}/acknowledge
```

```
GET  /api/logs?signed=true&from=&to=    (export légal)
WS   /ws/events                         (stream temps réel)
```

---

9. Flux de données de bout en bout

Scénario nominal — Accès badge normal

```
09:02:14  Badge #1042 scanné → Porte A3 (zone serveurs)
          → MQTT: building/B1/zone/Z3/badge/R07
          → Gateway: format unifié JSON
          → Middleware Node-RED: normalisation, enrichissement user "alice@corp"
          → Kafka topic events.raw
          → IA: profil alice = accès Z3 attendu 8h-18h, score physique = 0.05
          → Score réseau : trafic alice normal, score cyber = 0.03
          → Score final = 0.04 → NORMAL
          → TimescaleDB: log signé (ECC-hybrid-MLDSA5)
          → Dashboard: mise à jour statut zone Z3 (vert)
```

Scénario d'attaque hybride — Intrusion masquée

```
03:17:42  Porte B2 (zone data center) ouverte sans badge associé
          → Door sensor: state=FORCED, no_badge_in_window=true
          → Score physique partiel: 0.80 → CRITICAL

03:17:45  Trafic réseau inhabituel depuis l'IP caméra B2-CAM-01
          → Scan de ports internes détecté
          → Score cyber: 0.80

03:17:47  Corrélation: même zone B2, fenêtre temporelle 3s
          → malus corrélation: +0.20
          → Score final: min(1.0, 0.80×0.5 + 0.80×0.5 + 0.20) = 1.0
          → CRITICAL

          → Kafka topic alerts.critical
          → Notification SOC (SMS + dashboard rouge)
          → Action automatique possible: verrouillage porte B2, isolation VLAN caméra
          → Log signé (ECC-hybrid-MLDSA5) archivé dans TimescaleDB
```

---

10. Stack Technologique

> ***Infrastructure :*** *tout tourne sur* ***un seul PC*** *via* ***Docker Compose****. Chaque composant est un container isolé. Les communications inter-containers passent par le réseau Docker interne ; TLS est appliqué même en local pour valider l'implémentation crypto dans des conditions réalistes.*

Configuration matérielle cible

Ressource	Minimum recommandé	Rôle principal
CPU	8 cœurs	Kafka, IA (Isolation Forest), Node-RED
RAM	16 Go	Kafka + TimescaleDB + tous les services
Stockage	20 Go SSD	Logs TimescaleDB (séries temporelles)
OS	Windows 11 / macOS (ARM)	Docker Desktop (Engine 24+)

Services Docker Compose

```
# docker-compose.yml — vue simplifiée
services:
  simulator:        # Agents Python (badge, door, motion, network)
  mosquitto:        # MQTT Broker
  gateway:          # Service Python gateway (validation, TLS)
  nodered:          # Middleware / orchestration flows
  zookeeper:        # Requis par Kafka
  kafka:            # Streaming événements
  timescaledb:      # Base de données série temporelle
  ai-engine:        # Service Python : scoring IA
  backend:          # API FastAPI
  frontend:         # Dashboard React (servi par Nginx)
  grafana:          # Monitoring système
  prometheus:       # Collecte métriques
```

```
networks:
  iot-net:          # Réseau interne Docker (isolé)
```

Stack par couche

Couche	Composant	Langage / Runtime	Rôle
Simulation IoT	Agents Python	Python 3.12	Génération d'événements (badges, portes…)
Protocoles	Mosquitto 2.x	C	Broker MQTT
Gateway	Service Python asyncio	Python 3.12	Validation, buffer, publication MQTT
Sécurité TLS/PQC	OpenSSL 3.x + liboqs + oqs-python	Python 3.12	TLS 1.3 + X25519MLKEM768 + ECC-hybrid-MLDSA5
Middleware	Node-RED (self-hosted)	Node.js 20	Orchestration des flows IoT
Streaming	Apache Kafka	JVM	File de messages inter-services
Stockage	TimescaleDB	PostgreSQL 16	Séries temporelles + logs signés
IA Engine	Service Python	Python 3.12	scikit-learn (Isolation Forest)
API Backend	FastAPI	Python 3.12	REST + WebSocket dashboard
Frontend	React + TypeScript	Node.js 20 / Nginx	Dashboard SOC
Monitoring	Grafana + Prometheus	Go	Métriques système, latence pipeline

---

11. Modèle de données

11.1 Entités Métadonnées (Relationnel - Stockage standard)

```
[ Référentiel Statique ]              [ Flux d'Événements (Hypertable) ]

User ──────────┐                      ┌──────────────────────────────┐
  user_id (PK) │                      │          TABLE : events      │
  name         │       1:N            │   (Normalisation unifiée)    │
  clearance_lvl├──────────────────────┤                              │
  typical_zones│                      │  event_id (UUID)             │
               │                      │  timestamp (TIMESTAMPTZ)     │
Zone ──────────┤                      │  event_type (ENUM)           │
  zone_id (PK) │       1:N            │  source_layer (PHYS/CYBER)   │
  building_id  ├──────────────────────┤  building_id                 │
  risk_level   │                      │  zone_id (FK -> Zone)        │
               │                      │  device_id (FK -> Device)    │
Device ────────┤                      │  user_id (FK -> User)        │
  device_id (PK)│                     │  ai_score (FLOAT)            │
  zone_id (FK) │       1:N            │  ai_class (ENUM)             │
  type         ├──────────────────────┤  payload (JSONB)             │
  status       │                      │  signature (BYTEA)           │
               │                      └──────────────────────────────┘
Door ──────────┘                        ▲
  door_id (PK)                          │
  zone_id (FK) ──────── 1:N ────────────┘
  type
```

11.2 Détails des relations

- **Jointures IA** : Le moteur IA consomme la table `events` et fait des jointures avec `User` pour comparer l'événement actuel aux `typical_zones` et `typical_hours`.
- **Richesse du Payload** : Les données spécifiques (ex: `state` d'une porte, `access_result` d'un badge) sont stockées dans le champ `JSONB` pour garder une table principale flexible.
- **Partitionnement temporel (Hypertables)** : Pour maintenir des hautes performances sur 15 ans, le stockage utilise le mécanisme d'hypertables. Contrairement à une table classique qui sature avec le volume, les données sont ici

découpées en **partitions physiques autonomes (“chunks”)** sur le disque dur. Chaque chunk représente une fenêtre temporelle (ex: 7 jours).

- *Bénéfice technique* : Lors d’une requête, le moteur de base de données cible uniquement les fichiers concernés (exclusion de partition), ce qui limite les entrées/sorties disque (I/O). Cela garantit que les index restent “chauds” (en RAM) et permet une suppression instantanée des données obsolètes par simple drop de fichier, sans fragmentation de la base.

---

12. Scénarios d’attaque & réponses attendues

Scénario	Signaux détectés	Score attendu	Réponse système
Accès badge hors horaires	Badge hors plage autorisée	0.55 SUSPECT	Alerte SOC niveau 2
Porte forcée (sans badge)	Door FORCED + no badge	0.80 CRITICAL	Alerte immédiate
Tailgating (2 personnes, 1 badge)	Motion > 1 personne + 1 badge	0.60 SUSPECT	Alerte + vérification caméra
Badge révoqué utilisé	Badge DENIED + tentative répétée	0.80 CRITICAL	Alerte + log légal
Intrusion physique + scan réseau	Door + mouvement + trafic anormal	1.0 CRITICAL	Alerte + isolation VLAN
Compromission caméra IoT	Trafic anormal depuis IP caméra	0.70 CRITICAL	Isolation device + alerte
Vol de credentials réseau après accès physique	Badge OK + trafic exfiltration post-accès	0.85 CRITICAL	Alerte corrélée

---

13. Défis techniques & points à trancher

Points ouverts (à décider en équipe)

- [x] **Simulation vs matériel réel :** tout simulé sur PC — simulateur Python multi-agents ✓
- [x] **Déploiement :** Docker Compose sur PC unique (Windows/Mac) ✓
- [ ] **Modèle IA :** Isolation Forest en v1 → LSTM en v2 si temps disponible
- [ ] **Implémentation PQC :** `liboqs` + `oqs-python` — X25519MLKEM768 + ECC-hybrid-MLDSA5 → **périmètre Ryan**
- [ ] **Stockage clés (simulation) :** fichiers `.pem` chiffrés AES-256 avec passphrase → à définir par Ryan
- [ ] **GDPR :** simulation de métadonnées uniquement, pas de vidéo réelle — problème écarté ✓
- [ ] **Signature des certs devices :** ECC-hybrid-MLDSA5 (cohérent avec les logs) ou ML-DSA-65 (plus léger) ? → à trancher par Ryan

Risques identifiés

Risque	Probabilité	Impact	Mitigation
Latence > 1s end-to-end	Faible	Élevé	Tout sur le même PC → réseau Docker interne rapide. Le **Session Resumption (PSK+DHE)** réduit l'overhead PQC de 90% après le 1er handshake.
Faux positifs trop élevés	Élevée	Moyen	Calibration seuils + feedback opérateur

Risque	Probabilité	Impact	Mitigation
Dérive du modèle IA	Moyenne	Élevé	Réentraînement automatique + alertes métriques
Saturation RAM (Kafka + TimescaleDB)	Moyenne	Élevé	Limites Docker par container, monitoring Grafana
Complexité Docker Compose (10+ services)	Moyenne	Moyen	Scripts de démarrage, healthchecks configurés

---

Structure du dépôt (proposition)

```
iot-security/
├── docs/
│   └── ARCHITECTURE.md          ← ce fichier
├── simulator/                    ← couche 1 : simulation des capteurs IoT
│   ├── agents/                  ← badge_agent.py, door_agent.py, etc.
│   ├── scenarios/               ← normal_day.py, hybrid_attack.py, etc.
│   ├── config.yaml              ← topologie bâtiment simulé
│   └── main.py                  ← orchestrateur du simulateur
├── gateway/                      ← couche 3 : gateway logiciel
│   └── gateway.py               ← validation, buffer, publication MQTT
├── security/                     ← couche 4 : TLS/PQC — périmètre Ryan
│   ├── ca/                      ← autorité racine (hybride)
│   ├── gateway/                 ← certificats & clés Gateway (hybride)
│   ├── middleware/              ← certificats & clés Middleware (hybride)
│   ├── tls_client.py            ← client TLS (X25519MLKEM768)
│   ├── tls_server.py            ← serveur TLS
│   └── log_signer.py            ← signature ECC-hybrid-MLDSA5 des logs
├── middleware/                   ← couche 5 : Node-RED flows
│   └── flows/                   ← fichiers flows.json exportés Node-RED
├── ai-engine/                    ← couche 7 : moteur IA
│   ├── models/                  ← modèles entraînés sérialisés
│   ├── training/                ← scripts d'entraînement Isolation Forest
│   └── scoring_service.py       ← Moteur d'analyse temps réel (Consommateur Kafka)
├── backend/                      ← couche 8 : API FastAPI
│   └── app/
├── frontend/                     ← couche 8 : dashboard React
│   └── src/
```

```
├── infra/
│   ├── docker-compose.yml        ← orchestre tous les services
│   ├── mosquitto/
│   │   └── mosquitto.conf
│   ├── kafka/
│   └── timescaledb/
│       └── init.sql              ← création des tables + hypertables
└── README.md
```

---

14. Estimation de la charge de travail (Workload)

Une estimation de la répartition du temps de développement par grand pôle technique :

Thème de travail	Charge (%)	Description
Sécurité & Cryptographie PQC	30%	Handshake hybride, Session Resumption PSK+DHE, Double Proxy Terminations.
Architecture Réseau & Docker	15%	Segmentation par Docker Networks isolés (internal:true), orchestration.
Moteur IA & Scoring	15%	Feature engineering, Isolation Forest, logique de fusion des scores.
Middleware & Pipeline Kafka	15%	Configuration Kafka, flows Node-RED, normalisation unifiée.
Dashboard SOC & Visualisation	15%	Interface React, WebSockets temps réel, cartographie des alertes.
Simulation IoT & Scénarios	10%	Développement des agents Python et des scripts d'injection d'attaques.

---

15. Répartition des rôles et responsabilités

Membre de l'équipe	Rôle principal	Missions clés	Charge (%)
Ryan ZERHOUNI	**Lead Sécurité & PQC**	Handshake hybride, Session Resumption, Double Proxy, signature PQC.	30%
(À définir)	**Expert IA & Scoring**	Feature engineering, modèle Isolation Forest et fusion des scores.	15%
(À définir)	**Lead Infra & Simulation**	Orchestration Docker, isolation réseaux (`internal: true`), simulateur.	25%
(À définir)	**Data Engineer**	Flows Node-RED, normalisation Unified Schema et Kafka.	15%
(À définir)	**Lead Frontend & SOC**	Dashboard React, WebSockets temps réel et cartographie SOC.	15%

Document mis à jour le 22/04/2026 — A valider par l'équipe, proposition faites par Ryan ZERHOUNI.